

Grundlagen der Programmierung:

11. Klausurvorbereitung I

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

1. Drei Puzzler zu OOP und C
2. Beispielaufgaben aus vergangenen Klausuren
 1. C Pointer
 2. Java Verzweigungen
 3. Java Operatoren
3. Lösung schriftliches Übungsblatt 3
4. Fragestunde

PUZZLER ZU OOP UND C

- Unter welches OOP-Prinzip fällt das Überschreiben (override) von Methoden?
⇒ Polymorphismus
- Schreiben Sie (in **Java**) die Klasse `Waehrung`. Diese Klasse beinhaltet
 - die private Instanzvariable **double** `value`
 - den öffentlichen Konstruktor mit einem Argument vom Typ **double**, um `value` zu initialisieren
 - die öffentliche Methode **double** `dollarValue()`. Diese gibt nur den Wert `value` zurück.
- Schreiben Sie eine zweite Klasse `Euro` die von `Waehrung` erbt. Die Klasse `Euro` soll
 - die Methode `dollarValue` so überschreiben, dass der zurück gegebene Betrag das 1,09-fache von `value` aus der Elternklasse ist.
 - der Faktor 1,09 soll als unveränderbare Konstante `EXCHANGE_RATE` in der Klasse `Euro` enthalten sein
 - den Standard-Konstruktor nutzen, der `value` mit 0 initialisiert

```
1 public class Waehrung {
2     private double value;
3
4     public Waehrung(double value) {
5         this.value = value;
6     }
7
8     public double dollarValue() {
9         return this.value;
10    }
11 }
```

```
1 public class Euro extends Waehrung {
2
3     private final double EXCHANGE_RATE = 1.09;
4
5     public Euro() {
6         super(0.0);
7     }
8
9     @Override
10    public double dollarValue() {
11        return EXCHANGE_RATE * super.dollarValue();
12    }
13 }
```

- Was ist der Unterschied der Speicherallokierung in den folgenden Beispielen?

```
1 int an_array[3] = {1, 2, 3};
```

Stack

- Wird nach dem Anfang/Ende einer Funktion *automatisch reserviert/freigegeben*.

```
1 int* an_array = malloc(sizeof(int) * 3);
```

Heap

- Muss *manuell* über die Funktion *free* *freigegeben* und über *malloc* *reserviert* werden.
- Größe dynamisch zur Laufzeit festlegbar.

- Erklären Sie was der **hervorgehobene** Operator in den folgenden Beispielen macht:

```
1 int x = 0;
2 int y = 3;
3
4 int z = x + y;
5 int a = --x;
6 int* p0 = &x; int* p1 = &y;
7 int* p2 = p0++;
8 int c = *p0 * *p1;
9 int b = *p0 * *p1;
10 int d = y << 3;
```

Addition der beiden Variableninhalte x und y.

Entspricht $x=x-1$. Die Subtraktion findet vor der Zuweisung statt.

Referenziert (liefert Speicheradresse der Variable) x bzw. y.

Addiert (nach Zuweisung) `sizeof(int)` auf die Speicheradresse p0.

Dereferenziert p1 (gibt den Wert an der Adresse in p1 zurück).

Multipliziert den linken und rechten Wert miteinander.

Arithmetischer Links-Shift um 3. Entspricht Multiplikation von y mit 2^3 (8).

Sind folgende Aussagen über **C structs** wahr oder falsch?

-  1. **structs** gruppieren Variablen miteinander
-  2. Ähnlich wie Klassen können **structs** Methoden enthalten
-  3. **structs** müssen mit `malloc` initialisiert werden
-  4. **structs** sind nutzerdefinierte Datentypen
-  5. Die Vererbung von **structs** ist möglich
-  6. Nach der Deklaration kann das Schlüsselwort **struct** weggelassen werden

- Pointer unterliegen eigener Arithmetik

```
1 int    a = 70;
2 int*   b = (int*) 70;
3 char   c = 70;
4 char*  d = (char*) 70;
5
6 a++; b++; c++; d++;
7
8 printf("value of a: %d \n", a);
9 printf("value of b: %d \n", b);
10 printf("value of c: %c \n", c);
11 printf("value of d: %d \n", d);
12
13 printf("size of a: %d \n", sizeof(a));
14 printf("size of b: %d \n", sizeof(b));
15 printf("size of c: %d \n", sizeof(c));
16 printf("size of d: %d \n", sizeof(d));
```

```
71
74
G (== 71)
71

4
4 bzw. 8
1
4 bzw. 8
```

BEISPIELAUFGABEN AUS VERGANGENEN KLAUSUREN

Klausurbeispielaufgaben

C POINTER

C Pointer – 8 Punkte

Es ist folgende **struct** gegeben:

```
1 struct foo {  
2     int a;  
3     int b;  
4 };
```

Weiterhin ist folgende Funktion definiert:

```
5 int * aTob(int * a) {  
6     // diese Funktion tut etwas  
7 }
```

Diese Funktion nimmt einen Integer-Pointer als Parameter entgegen, berechnet irgendetwas und liefert einen Pointer auf eine neue Integer-Variable zurück. Die tatsächliche Berechnung innerhalb der Funktion ist im Rahmen dieser Aufgabe irrelevant.

C Pointer – 8 Punkte

Es gelten für allen Code in dieser Aufgabe die folgenden Include-Anweisungen:

```
1 #include <stdlib.h>
2 #include <stdio.h>
```

C Pointer – 8 Punkte

Deklariieren und implementieren Sie eine Funktion namens `initFoo()`. Diese soll eine neue Instanz der **struct** `foo` instantiieren und einen Pointer auf diese zurückgeben. Zusätzlich soll sie folgende Initialwerte für die Attribute (members) von `foo` setzen:

4 Punkte

- `a` soll mit `5` initialisiert werden.
- `b` soll mit `0` initialisiert werden

`initFoo()` benötigt keine Eingabe-Parameter.

```
1  
2  
3  
4  
5  
6
```

C Pointer – 8 Punkte

Deklariert und implementiert Sie eine Funktion namens `initFoo()`. Diese soll eine neue Instanz der `struct foo` instantiieren und einen Pointer auf diese zurückgeben. Zusätzlich soll sie folgende Initialwerte für die Attribute (members) von `foo` setzen:

4 Punkte

- `a` soll mit `5` initialisiert werden.
- `b` soll mit `0` initialisiert werden

`initFoo()` benötigt keine Eingabe-Parameter.

```
1 struct foo* initFoo() {  
2     struct foo *result = malloc(sizeof(struct foo));  
3     result->a = 5;  
4     result->b = 0;  
5     return result;  
6 }
```

C Pointer – 8 Punkte

Vervollständigen Sie die main-Funktion.

4 Punkte

Sie soll mit Hilfe von `initFoo()` eine neue Instanz der **struct** `foo` erstellen. Dann soll sie den Wert des `a` Attributs, welches in `initFoo()` initialisiert worden ist, an die Funktion `aTob(int *a)` übergeben und den Wert der zurückgegebenen Variable in das `b` Attribut von `foo` schreiben. Nehmen Sie, falls nötig, Typ- und/oder Pointerkonvertierungen vor.

Vervollständigen Sie abschließend die Konsolenausgaben der beiden Attribute der **struct** `foo` Instanz.

C Pointer – 8 Punkte

```
1  int main() {
2      /*
3       * Initialisieren Sie hier eine Instanz von foo mit dem
4       * Aufruf von initFoo()
5       */
6
7
8      /*
9       * uebergeben Sie hier foos a Attribut an aTob und schreiben
10      * Sie die Rueckgabe in foos b Attribut
11      */
12
13
14     /*
15      * Konsolenausgaben. Uebergeben Sie hier die Werte von foos
16      * Attributen jeweils an den 2. Parameter des printf-Aufrufs
17      */
18     printf("a = %d\n",      );
19     printf("b = %d\n",      );
20 }
```

C Pointer – 8 Punkte

```
1 int main() {
2     /*
3      * Initialisieren Sie hier eine Instanz von foo mit dem
4      * Aufruf von initFoo()
5      */
6     struct foo *afoo = initFoo();
7
8     /*
9      * uebergeben Sie hier foos a Attribut an aTob und schreiben
10     * Sie die Rueckgabe in foos b Attribut
11     */
12
13
14     /*
15     * Konsolenausgaben. Uebergeben Sie hier die Werte von foos
16     * Attributen jeweils an den 2. Parameter des printf-Aufrufs
17     */
18     printf("a = %d\n",      );
19     printf("b = %d\n",      );
20 }
```

C Pointer – 8 Punkte

```
1 int main() {
2     /*
3      * Initialisieren Sie hier eine Instanz von foo mit dem
4      * Aufruf von initFoo()
5      */
6     struct foo *afoo = initFoo();
7
8     /*
9      * uebergeben Sie hier foos a Attribut an aTob und schreiben
10     * Sie die Rueckgabe in foos b Attribut
11     */
12     afoo->b = *aTob(&afoo->a);
13
14     /*
15     * Konsolenausgaben. Uebergeben Sie hier die Werte von foos
16     * Attributen jeweils an den 2. Parameter des printf-Aufrufs
17     */
18     printf("a = %d\n",      );
19     printf("b = %d\n",      );
20 }
```

C Pointer – 8 Punkte

```
1 int main() {
2     /*
3      * Initialisieren Sie hier eine Instanz von foo mit dem
4      * Aufruf von initFoo()
5      */
6     struct foo *afoo = initFoo();
7
8     /*
9      * uebergeben Sie hier foos a Attribut an aTob und schreiben
10     * Sie die Rueckgabe in foos b Attribut
11     */
12     afoo->b = *aTob(&afoo->a);
13
14     /*
15     * Konsolenausgaben. Uebergeben Sie hier die Werte von foos
16     * Attributen jeweils an den 2. Parameter des printf-Aufrufs
17     */
18     printf("a = %d\n", afoo->a);
19     printf("b = %d\n", afoo->b);
20 }
```

Klausurbeispielaufgaben

JAVA VERZWEIGUNGEN

Java Verzweigungen – 11 Punkte

In dieser Aufgabe implementieren Sie folgende Methode um in Java ein Tier anhand einer **id** zu erkennen:

```
1 public static void printAnimal(int id);
```

In der Klasse der Methode sind folgende Konstanten definiert:

```
2 static final int DOG = 0;  
3 static final int CAT = 1;  
4 static final int COW = 2;  
5 static final int PIG = 3;
```

Die Methode soll anhand der überwiesenen ID unterscheiden und dann eine passende Nachricht auf die Konsole ausgeben (z. B. "Dog" oder "Cat"). Im Fall einer ungültigen ID, soll ein Fehler ausgegeben werden.

Verwenden Sie bei der Implementierung keine Integer-Literale („1“, „2“, etc.), sondern benutzen Sie ausschließlich die Konstanten.

Java Verzweigungen – 11 Punkte

Implementieren Sie die `printAnimal`-Methode mit `if` und `else`-Konstrukten.
Schachteln Sie keine `ifs`

2 Punkte

Java Verzweigungen – 11 Punkte

Implementieren Sie die printAnimal-Methode mit **if** und **else**-Konstrukten.
Schachteln Sie keine **ifs**

2 Punkte

```
1 public static void printAnimal(int id) {  
2     String result = "";  
3     if (id == DOG) result = "Dog";  
4     else if (id == CAT) result = "Cat";  
5     else if (id == COW) result = "Cow";  
6     else if (id == PIG) result = "Pig";  
7     else result = "Invalid ID!";  
8     System.out.println(result);  
9 }
```

Java Verzweigungen – 11 Punkte

Implementieren Sie die `printAnimal`-Methode mit `switch` und `case`-Konstrukten.

3 Punkte

Hinweis: Um Schreibarbeit zu sparen, dürfen sie in dieser Teilaufgabe den Code für die Konsolenausgabe beliebig abkürzen, z.B. mit `sout(...)` oder `print(...)`. Es muss nur erkennbar sein, dass eine Konsolenausgabe stattfindet.

Java Verzweigungen – 11 Punkte

Implementieren Sie die printAnimal-Methode mit switch und case-Konstrukten.

3 Punkte

Hinweis: Um Schreibarbeit zu sparen, dürfen sie in dieser Teilaufgabe den Code für die Konsolenausgabe beliebig abkürzen, z.B. mit sout(...) oder print(...). Es muss nur erkennbar sein, dass eine Konsolenausgabe stattfindet.

```
1 public static void printAnimal(int id) {  
2     String result = "";  
3     switch (id) {  
4         case DOG: result = "Dog"; break;  
5         case CAT: result = "Cat"; break;  
6         case COW: result = "Cow"; break;  
7         case PIG: result = "Pig"; break;  
8         default: result = "Invalid ID!";  
9     }  
10    System.out.println(result);  
11 }
```

Java Verzweigungen – 11 Punkte

Definieren Sie ein **enum** und verwenden Sie dieses anstelle der Konstanten. Verändern Sie dazu die Deklaration der Methode und implementieren Sie diese. Sie haben die freie Wahl an Verzweigungs-Konstrukten. Vergessen Sie nicht auch einen Fall für ein unspezifiziertes Tier (s. die Fehlermeldung in den vorherigen Aufgabenteilen) bereitzustellen.

4 Punkte

Hinweis: Um Schreibarbeit zu sparen, dürfen sie in dieser Teilaufgabe den Code für die Konsolenausgabe beliebig abkürzen, z.B. mit `sout(...)` oder `print(...)`. Es muss nur erkennbar sein, dass eine Konsolenausgabe stattfindet.

Java Verzweigungen – 11 Punkte

4 Punkte

```
1 public enum Animal {
2     DOG, CAT, COW, PIG, UNICORN
3 }
4
5 public static void printAnimal(Animal animal) {
6     String result = "";
7     switch (animal) {
8         case DOG: result = "Dog"; break;
9         case CAT: result = "Cat"; break;
10        case COW: result = "Cow"; break;
11        case PIG: result = "Pig"; break;
12        default: result = "Unspecified mythical creature!";
13    }
14    System.out.println(result);
15 }
```

Java Verzweigungen – 11 Punkte

Erläutern Sie kurz stichpunktartig die Vorteile von `enums` gegenüber Konstanten.

2 Punkte

Erläutern Sie kurz stichpunktartig die Vorteile von `enums` gegenüber Konstanten.

2 Punkte

➤ **Begrenzte Zahl an Varianten**

Anstelle der `int`-Konstanten kann eine beliebige Zahl übergeben werden. Im Fall von `enums` muss eine der vorher festgelegten Varianten übergeben werden.

Erläutern Sie kurz stichpunktartig die Vorteile von `enums` gegenüber Konstanten.

2 Punkte

➤ **Begrenzte Zahl an Varianten**

Anstelle der `int`-Konstanten kann eine beliebige Zahl übergeben werden. Im Fall von `enums` muss eine der vorher festgelegten Varianten übergeben werden.

➤ **Typsicherheit**

Ist ein `enum` gefordert, das Tiere darstellt, kann nicht eines übergeben werden, das die Monate aufzählt. Für `int`-Konstanten ist dies nicht gegeben.

Erläutern Sie kurz stichpunktartig die Vorteile von **enums** gegenüber Konstanten.

2 Punkte

➤ **Begrenzte Zahl an Varianten**

Anstelle der **int**-Konstanten kann eine beliebige Zahl übergeben werden. Im Fall von **enums** muss eine der vorher festgelegten Varianten übergeben werden.

➤ **Typsicherheit**

Ist ein **enum** gefordert, das Tiere darstellt, kann nicht eines übergeben werden, das die Monate aufzählt. Für **int**-Konstanten ist dies nicht gegeben.

➤ **enums sind Klassen**

enums dürfen Attribute und Methoden haben und können Interfaces implementieren.

Klausurbeispielaufgaben

JAVA OPERATOREN

Java Operatoren – 5 Punkte

Diese Aufgabe prüft ihr Verständnis von Operatoren in Java.

3 Punkte

Es sei `arg` ein Objekt vom Typ `String`.

Folgende Bedingung ist als Teil einer `while`-Schleife gegeben:

```
1 while (arg != null & arg != "done") {  
2     // hier kommt Code, der arg veraendert  
3 }
```

Java Operatoren – 5 Punkte

1. Was passiert, bei der Abfrage der Bedingung, wenn arg gleich null ist und warum? **3 Punkte**

2. Was passiert bei der Abfrage der Bedingung, wenn der Inhalt von arg gleich "done" ist und warum?

Java Operatoren – 5 Punkte

1. Was passiert, bei der Abfrage der Bedingung, wenn `arg` gleich `null` ist und warum? **3 Punkte**

Der Ausdruck `arg != null` ist **false** und damit auch das Ergebnis der Bedingung. Durch den einfachen `&`-Operator wird der zweite Ausdruck `arg != "done"` trotzdem ausgewertet. Da wir aber lediglich einen Referenzvergleich durchführen, passiert nichts schlimmes. Hätten wir z. B. `equals` verwendet wäre an dieser Stelle eine `NullPointerException` geworfen worden.

2. Was passiert bei der Abfrage der Bedingung, wenn der Inhalt von `arg` gleich **"done"** ist und warum?

Java Operatoren – 5 Punkte

1. Was passiert, bei der Abfrage der Bedingung, wenn `arg` gleich `null` ist und warum? **3 Punkte**

Der Ausdruck `arg != null` ist **false** und damit auch das Ergebnis der Bedingung. Durch den einfachen `&`-Operator wird der zweite Ausdruck `arg != "done"` trotzdem ausgewertet. Da wir aber lediglich einen Referenzvergleich durchführen, passiert nichts schlimmes. Hätten wir z. B. `equals` verwendet wäre an dieser Stelle eine `NullPointerException` geworfen worden.

2. Was passiert bei der Abfrage der Bedingung, wenn der Inhalt von `arg` gleich **"done"** ist und warum?

Der Ausdruck `arg != null` ist **true**. Der Wert des Ausdrucks `arg != "done"` ist nicht vorhersagbar, da kein `equals` verwendet wurde. Damit ist der Wert des Gesamtausdrucks auch unvorhersehbar.

Java Operatoren – 5 Punkte

Kreuzen Sie an, ob der jeweils angegebene, in Java spezifizierte boolesche Ausdruck **true** oder **false** zurückgibt. Die Ausdrücke werden in absteigender Reihenfolge bearbeitet. D. h. potentielle Änderungen von Variablenwerten wirken sich auf die folgenden Ausdrücke aus. **2 Punkte**

Hinweis: Falsche Kreuze führen zu Punktabzug.

```
int i = 5;
```

Ausdruck	true	false
<code>8 > i & (4 % 3) == 1</code>		
<code>i >= 5 i++ >= 4</code>		
<code>i > 5 (i = 8) <= 7</code>		
<code>i > 7 ? false i / 2 != 4 : (2 >> 1) > 0</code>		

Java Operatoren – 5 Punkte

Kreuzen Sie an, ob der jeweils angegebene, in Java spezifizierte boolesche Ausdruck **true** oder **false** zurückgibt. Die Ausdrücke werden in absteigender Reihenfolge bearbeitet. D. h. potentielle Änderungen von Variablenwerten wirken sich auf die folgenden Ausdrücke aus. **2 Punkte**

Hinweis: Falsche Kreuze führen zu Punktabzug.

`int i = 5;`

Ausdruck	true	false
<code>8 > i & (4 % 3) == 1</code>	✗	
<code>i >= 5 i++ >= 4</code>		
<code>i > 5 (i = 8) <= 7</code>		
<code>i > 7 ? false i / 2 != 4 : (2 >> 1) > 0</code>		

`i = 5`

Java Operatoren – 5 Punkte

Kreuzen Sie an, ob der jeweils angegebene, in Java spezifizierte boolesche Ausdruck **true** oder **false** zurückgibt. Die Ausdrücke werden in absteigender Reihenfolge bearbeitet. D. h. potentielle Änderungen von Variablenwerten wirken sich auf die folgenden Ausdrücke aus. **2 Punkte**

Hinweis: Falsche Kreuze führen zu Punktabzug.

`int i = 5;`

Ausdruck	true	false	
8 > i & (4 % 3) == 1	✖		i = 5
i >= 5 i++ >= 4	✖		i = 5
i > 5 (i = 8) <= 7			
i > 7 ? false i / 2 != 4 : (2 >> 1) > 0			

Java Operatoren – 5 Punkte

Kreuzen Sie an, ob der jeweils angegebene, in Java spezifizierte boolesche Ausdruck **true** oder **false** zurückgibt. Die Ausdrücke werden in absteigender Reihenfolge bearbeitet. D. h. potentielle Änderungen von Variablenwerten wirken sich auf die folgenden Ausdrücke aus. **2 Punkte**

Hinweis: Falsche Kreuze führen zu Punktabzug.

`int i = 5;`

Ausdruck	true	false	
<code>8 > i & (4 % 3) == 1</code>	✗		<code>i = 5</code>
<code>i >= 5 i++ >= 4</code>	✗		<code>i = 5</code>
<code>i > 5 (i = 8) <= 7</code>		✗	<code>i = 8</code>
<code>i > 7 ? false i / 2 != 4 : (2 >> 1) > 0</code>			

Java Operatoren – 5 Punkte

Kreuzen Sie an, ob der jeweils angegebene, in Java spezifizierte boolesche Ausdruck **true** oder **false** zurückgibt. Die Ausdrücke werden in absteigender Reihenfolge bearbeitet. D. h. potentielle Änderungen von Variablenwerten wirken sich auf die folgenden Ausdrücke aus. **2 Punkte**

Hinweis: Falsche Kreuze führen zu Punktabzug.

`int i = 5;`

Ausdruck	true	false	
<code>8 > i & (4 % 3) == 1</code>	✗		<code>i = 5</code>
<code>i >= 5 i++ >= 4</code>	✗		<code>i = 5</code>
<code>i > 5 (i = 8) <= 7</code>		✗	<code>i = 8</code>
<code>i > 7 ? false i / 2 != 4 : (2 >> 1) > 0</code>		✗	<code>i = 8</code>